

Task 1 : Group Anagrams

Aim

To group strings that are anagrams of each other using Java Stream API and `Collectors.groupingBy()`.

Algorithm

Read the array of strings.

Convert the array into a stream.

For each string:

Convert it into a character array.

Sort the characters.

Convert the sorted characters back into a string.

Use the sorted string as the key in `Collectors.groupingBy()`.

Collect all strings having the same key into a list.

Return all grouped anagrams.

Program :

```
import java.util.*;
```

```
import java.util.stream.*;
```

```
public class GroupAnagrams {
```

```
    public static List<List<String>> groupAnagrams(String[] strs) {
```

```
        return new ArrayList<>(  
            Arrays.stream(strs)
```

```
                .collect(Collectors.groupingBy(s -> {
```

```
                    char[] ch = s.toCharArray();
```

```
                    Arrays.sort(ch);
```

```
                })
```

```

        return new String(ch);
    }
    .values()
);
}

public static void main(String[] args) {
    String[] strs = {"eat", "tea", "tan", "ate", "nat", "bat"};

    List<List<String>> result = groupAnagrams(strs);

    System.out.println(result);
}
}

```

Output :

```
[[eat, tea, ate], [tan, nat], [bat]]
```

Result

The program successfully groups all strings that are anagrams of each other using Java Stream API.

Task 2 : Top K Frequent Elements

Aim

To find the K most frequent elements in an array using Java Stream API.

Algorithm

Read the integer array and value of k.

Convert the array into a stream.

Box primitive integers into Integer objects.

Count the frequency of each element using `Collectors.groupingBy()` and `Collectors.counting()`.

Convert the frequency map into a stream.

Sort the entries in descending order of frequency.

Select the first k elements using `limit(k)`.

Convert the result into an integer array.

Display the output.

Program :

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

public class TopKFrequent {

    public static int[] topKFrequent(int[] nums, int k) {

        Map<Integer, Long> freq =
            Arrays.stream(nums)
                .boxed()
                .collect(Collectors.groupingBy(
                    Function.identity(),
                    Collectors.counting()));
```

```

return freq.entrySet()
    .stream()
    .sorted((a, b) -> Long.compare(b.getValue(), a.getValue()))
    .limit(k)
    .mapToInt(Map.Entry::getKey)
    .toArray();
}

```

```

public static void main(String[] args) {

    int[] nums = {1, 1, 1, 2, 2, 3};
    int k = 2;

    int[] result = topKFrequent(nums, k);

    System.out.println(Arrays.toString(result));
}
}

```

Output :

[1, 2]

Result:

The program successfully finds the K most frequent elements from the given array using Java Stream API and pipeline processing.

Task 3 : Java Dequeue

Aim

To find the maximum number of unique integers present in any contiguous subarray (window) of size m using Java Deque and HashSet.

Algorithm

Read the values of n (number of integers) and m (window size).

Create a Deque<Integer> to store the current window.

Create a HashSet<Integer> to store unique elements in the window.

Traverse the array:

Add the current element to the deque.

If the deque size exceeds m, remove the first element.

Update the set with the elements in the current window.

Count the number of unique elements.

Keep track of the maximum unique count.

Display the maximum number of unique integers.

Program :

```
import java.util.*;

public class JavaDeque {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        Deque<Integer> deque = new ArrayDeque<>();
        Map<Integer, Integer> map = new HashMap<>();

        int max = 0;

        for (int i = 0; i < n; i++) {
            int num = sc.nextInt();
```

```

deque.addLast(num);
map.put(num, map.getOrDefault(num, 0) + 1);

if (deque.size() > m) {
    int removed = deque.removeFirst();
    map.put(removed, map.get(removed) - 1);

    if (map.get(removed) == 0)
        map.remove(removed);
}

if (deque.size() == m) {
    max = Math.max(max, map.size());
}
}

System.out.println(max);
}

```

}

Input

6 3

5 3 5 2 3 2

Sample Output

3

Result

The program successfully finds the maximum number of distinct integers present in any sliding window of size m using Java Deque and HashMap.

Task 4 : Java Hashset

Aim

To determine the number of distinct pairs of strings using Java HashSet.

Algorithm

Read the number of string pairs t.

Create a HashSet<String>.

For each pair:

Read the two strings.

Concatenate them into a single string separated by a space.

Insert the combined string into the HashSet.

Print the current size of the HashSet.

Continue until all pairs are processed.

Program :

```
import java.util.*;
```

```
public class JavaHashset {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        int t = sc.nextInt();  
  
        HashSet<String> set = new HashSet<>();  
  
        while (t-- > 0) {  
            String left = sc.next();  
            String right = sc.next();  
  
            set.add(left + " " + right);  
            System.out.println(set.size());  
        }  
    }  
}
```

Input :

5

john tom

john mary

john tom

mary anna

john mary

Sample Output

1

2

2

3

3

Result

The program successfully counts and displays the number of distinct string pairs after each insertion using Java HashSet.